

Zani's Eatery Backend Migration

A comprehensive solution architect presentation explaining how the restaurant moved from manual bookings and orders to an AWS-backed digital workflow. The main focus is the backend: authentication, APIs, business logic, database storage, staff notifications, security, monitoring, cost control and migration benefits.

24 slides

expanded version with more explanatory content

Backend first

challenges and solution focus on booking/order processing



The presentation follows the full backend migration story.

- | | | |
|----|--------------------------------|--|
| 01 | Company overview | Restaurant background, where it is from, why it was chosen, and what the owner wanted to improve. |
| 02 | Backend challenges | The operational problems caused by paper records, phone orders, local files and no central database. |
| 03 | AWS target architecture | How S3, Cognito, API Gateway, Lambda, RDS/DynamoDB and SNS work together. |
| 04 | Backend request flows | Detailed booking and order journeys from customer action to staff notification. |
| 05 | Security and operations | IAM, encryption, POPIA alignment, monitoring, backups, scaling and cost control. |
| 06 | Benefits and roadmap | Why AWS is a good fit and how the restaurant should continue the migration. |

Mission & Vision

Zani's Eatery aims to deliver quality comfort food while uplifting the local community through entrepreneurship, job creation, and authentic township dining experiences. The brand emphasizes community connection, accessibility, and consistency as key

Zani's Eatery is a township-based fast food and casual dining business founded in 2021 by couple and entrepreneurs Manini Mimi Mofokeng and Bongani Nkuta in Kattlehong, Gauteng. The company focuses on affordable, flavourful fast food and township-style meals, serving items such as burgers, wings, chips, wraps, grilled meats, kota-style meals, desserts, shakes, and beverages.

The eatery combines:

- Walk-in township dining
- Takeaway services
- Food delivery partnerships

Its success is strongly tied to community loyalty, consistent food quality and strategic use of delivery platforms such as Mr D Food and Uber Eats.

Company

- Zani's Eatery is a popular restaurant in Kattlehong, Gauteng.

Zani's Eatery is a local restaurant that needed a digital operating system.

Restaurant background

Zani's Eatery is presented as a growing local eatery in Kattlehong, South Africa. The restaurant receives a high volume of bookings and food orders from walk-in customers, phone calls and informal messages. Customers expect faster confirmation, accurate orders and a convenient way to interact with the restaurant online.

Before the migration, the owner relied on manual processes such as paper records, local spreadsheets, staff memory and phone conversations. That approach worked when demand was small, but it became unreliable as more customers started placing bookings and orders every day.

100+

daily customers noted in the project report

54.2K

social following used as a demand signal

Business objective

Create a simple static website and connect it to a backend that can authenticate customers, store bookings and orders, notify staff and reduce operational errors.

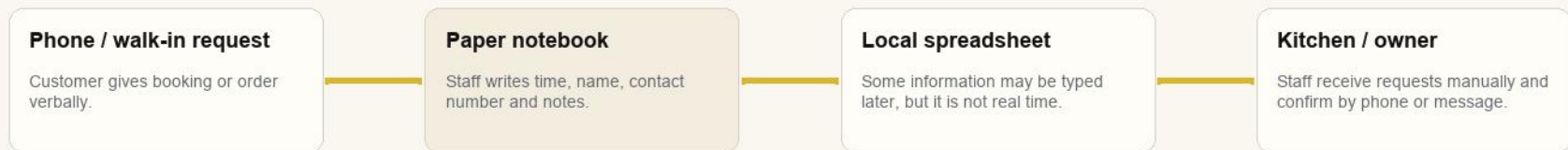
The restaurant was chosen because its manual process shows a clear cloud migration need.

- Bookings and orders are high-frequency activities, so mistakes happen often when there is no central backend.
- The restaurant depends on customer details, time slots, table availability, menu items, order totals and special requests.
- Those data points should not live in separate notebooks, phone messages or local files.
- A static website gives customers a front door; AWS services provide the backend workflow that makes the front door useful.



The project therefore becomes a practical example of cloud migration: replacing manual coordination with

Before AWS, the restaurant backend was people, paper and disconnected local records.



This manual process has no reliable system of record. Two people can accept the same booking, an order can be written incorrectly, a customer can be forgotten, and the owner cannot easily search customer history or measure demand patterns. In

The main challenges are backend problems, not only website problems.

Challenge	Detailed explanation	Backend impact
No cloud backend	The restaurant has not migrated bookings and orders to cloud services. Requests are handled through paper, phone calls, local files or staff memory.	There is no shared, reliable backend that all staff can use at the same time.
Order mix-ups	Handwritten or verbal orders can be misread, especially during rush periods. Wrong meals may be prepared or delivered.	The backend must validate menu items, quantities, customer details and order status.
Double bookings	Two staff members can accept bookings for the same table and time because there is no real-time availability check.	The backend must check existing bookings before confirming a reservation.
No customer history	The owner cannot easily identify repeat customers, preferences, allergies, special requests or previous order patterns.	The backend needs customer profiles linked to bookings and orders.
Costly scaling	Handling more customers manually means hiring more admin staff or buying local hardware, both of which are expensive.	Managed AWS services scale without the restaurant managing servers.

The migrated backend must become the restaurant's digital operations desk.

Customer identity

Know who is submitting a booking or order and protect their history from other users.

Persistent data

Store customer profiles, bookings, orders and order items in a managed database.

Security

Protect personal information using authentication, encryption, IAM roles and controlled API access.

Business validation

Check required fields, time slots, duplicate bookings, menu item IDs, prices and order totals.

Notifications

Alert staff immediately and send the customer confirmation after a valid request.

Operations

Monitor failures, back up data and scale during busy periods without managing servers.

The website is static, but the backend behind it is dynamic.

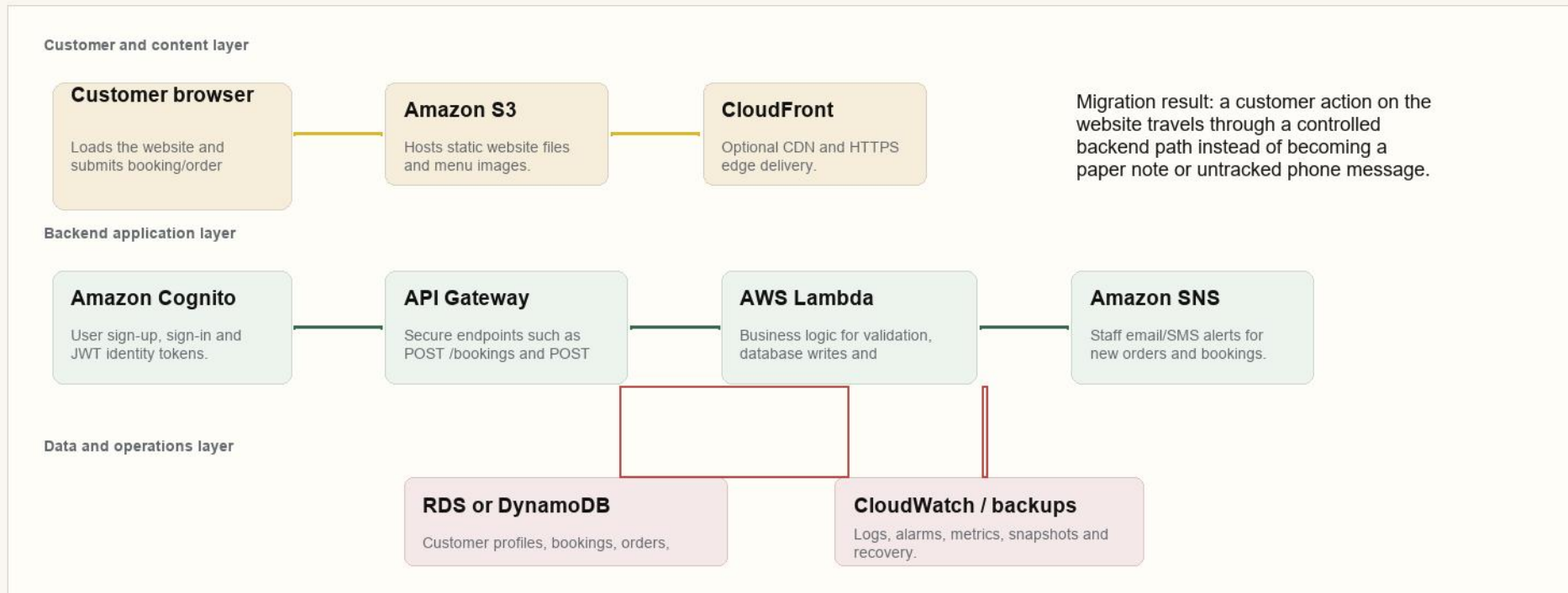


What the static website does

- Displays the restaurant brand, menu, navigation and ordering surfaces.
- Runs in the customer's browser after being served from Amazon S3.
- Collects user input through forms such as bookings, cart checkout and history pages.
- Calls backend APIs when a customer needs to create or retrieve real data.

Key point: S3 hosts the front end. It does not process bookings or orders. Processing moves to API Gateway, Lambda and the database.

The AWS target architecture separates presentation, backend logic, data and notifications.



Each AWS service owns a specific backend responsibility.

AWS service	Responsibility in this migration	Why the restaurant needs it
Amazon S3	Hosts the static website and stores menu photos and other image assets.	Customers need a reliable online front door without the owner running a web server.
Amazon Cognito	Manages customer registration, sign-in, tokens and protected history access.	The backend needs to know which customer created each booking or order.
API Gateway	Exposes secure endpoints for bookings, orders, customer history and menu data.	The website needs a controlled entry point into backend services.
AWS Lambda	Executes booking logic, order validation, database writes and confirmation triggers.	Business rules must run automatically when a customer submits a request.
Amazon RDS / DynamoDB	Stores profiles, bookings, orders, items, statuses and timestamps.	The restaurant needs a central source of truth instead of scattered records.
Amazon SNS	Sends staff alerts and customer confirmation messages.	Staff must know about new requests immediately.

Amazon S3 hosts the static website and stores image assets.

Amazon S3

S3 is object storage. In this project it stores static web files such as HTML, CSS, JavaScript and menu images. When a customer opens the restaurant website, the browser downloads those files from S3. This is cheaper and simpler than running a traditional web server.

Why it matters for the backend

- No server maintenance for the website layer.
- Stores menu photos and restaurant media as durable objects.
- Works well with CloudFront for faster delivery and HTTPS.
- Keeps frontend hosting separate from backend business logic.

Amazon Cognito provides customer identity and protected history access.

Amazon Cognito

Cognito manages customer sign-up and sign-in. After a successful login, it issues a secure token that proves who the customer is. The backend can use that token to link bookings and orders to the correct customer profile.

Why it matters for the backend

- Supports registration and login without custom password handling.
- Allows customers to access booking and order history.
- Helps protect personal data by requiring authentication.
- Provides identity information that Lambda can use safely.

API Gateway is the controlled front door into the backend.

API Gateway

The static website cannot write directly into the database. Instead, it calls API Gateway endpoints such as POST /bookings or POST /orders. API Gateway receives HTTPS requests, checks authorization, applies throttling and routes traffic to Lambda.

Why it matters for the backend

- Creates clean backend endpoints for the website.
- Integrates with Cognito authorization.
- Protects the backend with throttling and request validation.
- Separates public website delivery from private backend processing.

AWS Lambda runs the restaurant's backend business logic.

AWS Lambda

Lambda executes code only when an event happens. For this restaurant, separate Lambda functions can process bookings, process orders and send confirmations. The restaurant does not need to manage servers, operating systems or capacity planning.

Why it matters for the backend

- Validates booking date, time, party size and conflicts.
- Validates order items, quantities, prices and customer details.
- Writes clean records to RDS or DynamoDB.
- Publishes notification events to SNS after successful processing.

The database becomes the restaurant's central system of record.

Amazon RDS /

A managed database replaces paper records and local spreadsheets. RDS is a strong fit when bookings, customers and orders need relational structure. DynamoDB is an alternative when simple key-value access and serverless scale are preferred.

Why it matters for the backend

- Stores customer profiles and preferences.
- Stores bookings with time, party size, table and status.
- Stores orders, order items, totals and timestamps.
- Enables reporting on repeat customers and popular menu items.

SNS sends staff alerts and customer confirmations.

Amazon SNS

Once Lambda validates and saves a booking or order, SNS can publish a message to staff email or SMS. This closes the loop: the request is not only stored, it is also pushed to the people who must act on it.

Why it matters for the backend

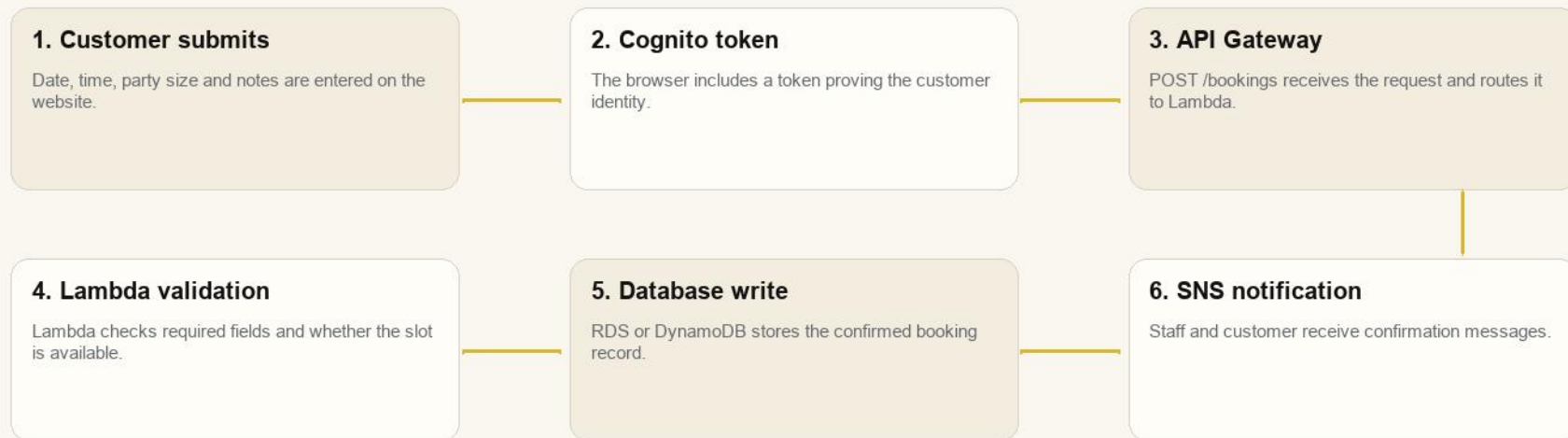
- Alerts staff immediately about new orders.
- Notifies staff about new bookings and special requests.
- Can support email, SMS or other subscribers.
- Reduces delays caused by manually checking messages.

The backend database stores structured records instead of scattered notes.

Entity	Fields stored	Why it matters
Customers	customer_id, name, phone, email, preferences, consent flag	Links every booking and order to a real customer profile.
Bookings	booking_id, customer_id, date, time, party_size, table, status, special_request	Prevents double bookings and creates a searchable reservation history.
Orders	order_id, customer_id, total, payment_status, order_status, timestamp	Gives the kitchen and owner a reliable order record.
OrderItems	order_item_id, order_id, menu_item_id, quantity, unit_price	Stores exactly what the customer ordered and supports reporting.
MenuItems	menu_item_id, name, price, category, availability	Allows the menu shown online to match items accepted by the backend.

This structure gives the owner a centralised view of customers, orders, bookings and menu demand. It also gives Lambda a reliable place to check availability and store validated requests.

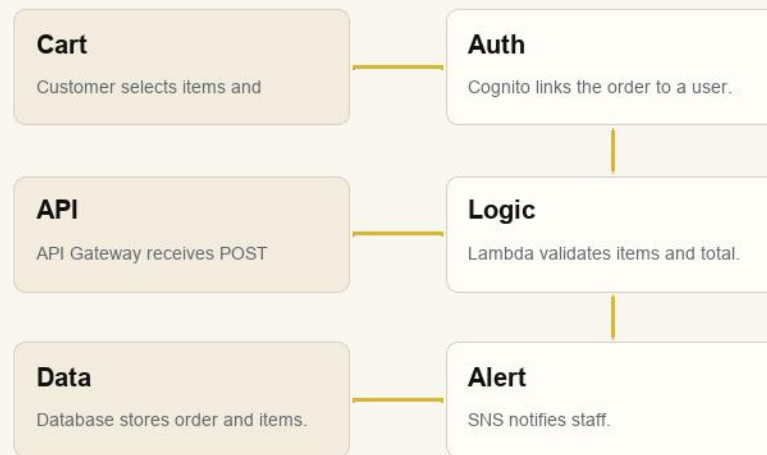
Booking flow: a reservation is authenticated, validated, stored and confirmed.



Order flow: cart items become a traceable restaurant order.

Manual order risk

- Phone orders can be misheard or written incorrectly.
- Kitchen instructions may be incomplete during peak hours.
- No order number means poor tracking and weak accountability.
- No history means the owner loses insight into customer behaviour.



Result: every order has a customer, item list, total, timestamp, status and staff notification trail.

The backend migration protects customer data through layered controls.

Identity

Cognito verifies customers and protects booking/order history.

API access

API Gateway validates requests, checks tokens and throttles suspicious traffic.

IAM permissions

Lambda functions use least-privilege roles for database and SNS access.

Data protection

Database encryption at rest, TLS in transit, backups and restricted network access protect records.

POPIA alignment

Collect only required data, manage consent and support deletion or correction requests.

The backend migration can be delivered in controlled phases.

Phase 1: Website foundation

Deploy the static website to S3 and store menu images as objects.

Phase 2: Identity

Configure Cognito user pool and connect sign-in to protected pages.

Phase 3: Database

Create RDS tables or DynamoDB collections for customers, bookings and orders.

Phase 4: APIs and Lambda

Build API Gateway routes and Lambda functions for booking and order processing.

Phase 5: Notifications

Connect Lambda to SNS so staff receive new request alerts.

Phase 6: Operations

Add CloudWatch logs, alarms, backups, IAM hardening and cost review.

AWS improves monitoring, recovery, scaling and cost visibility.

CloudWatch monitoring

Track Lambda errors, API latency, database CPU, failed notifications and unusual traffic.

Backups and recovery

Use RDS snapshots or DynamoDB point-in-time recovery to reduce data-loss risk.

Scaling

S3 and Lambda scale with demand, helping weekend peaks without buying servers.

Cost optimisation

Use pay-as-you-go pricing, Cognito free tier, right-sized database capacity and S3 storage classes.

R1,850

estimated AWS monthly cost from
project report

R8,500+

estimated monthly on-premises
comparison

AWS is the right fit because it solves the exact problems in the manual backend.

Accuracy

Validation reduces double bookings, wrong dishes and missing customer details.

Centralisation

Customer profiles, bookings and orders move into one managed source of truth.

Scalability

Serverless compute and managed storage handle demand without local hardware upgrades.

Speed

Staff receive notifications quickly and customers get faster confirmations.

Security

Cognito, IAM, encryption and API controls protect personal and business data.

Future growth

The same backend can support payments, loyalty, analytics and multiple branches.

The migration changes restaurant operations from reactive manual tracking to a managed cloud workflow.

- Amazon S3 hosts the static website and media so the restaurant has a reliable online presence.
- Amazon Cognito gives each customer a secure identity for bookings, orders and history.
- API Gateway and Lambda replace a traditional backend server with managed request handling and business logic.
- Amazon RDS or DynamoDB centralises customer profiles, booking details and order information.
- Amazon SNS closes the operational loop by notifying staff and confirming customer requests.
- Security, monitoring, backups and cost controls make the solution practical for a small but growing restaurant.

Final recommendation: keep the static website simple, but make the backend secure, event-driven and data-centred so the owner can reduce errors and scale confidently.